Material Suplementario — Métodos y Resultados Extendidos

Caracterización empírica de la conmutación por error de CloudNativePG en Kubernetes: estudio empírico de CloudNativePG bajo fallos inyectados (Fase 1)

Herrera, M. Parejo, A.
Universidad de Carabobo, Valencia, Venezuela
Correspondencia: Parejo, A. — ORCID 0009-0001-9737-7116

Nota. Este documento acompaña al artículo homónimo enviado a la Revista FARAUTE de Ciencias y Tecnología (UC/FACYT) y forma parte del depósito de reproducibilidad (Zenodo, DOI: 10.5281/zenodo.23004248). Recoge el detalle metodológico y estadístico que, por el límite de extensión del artículo, no cabe en el cuerpo principal, y documenta explícitamente los puntos que un árbitro necesita para auditar los resultados. Todas las cifras aquí y en el artículo se reproducen ejecutando `execution-package/manifiestos/scripts/analyze.py` sobre los CSV de `data/`.

S1 Propósito y alcance

La Fase 1 caracteriza empíricamente el comportamiento de CloudNativePG (CNPG) 1.28.0 bajo tres escenarios de fallo reproducibles de forma segura en un clúster productivo: F1 (*pod-kill*), F2 (*pod-failure*) y F3 (partición de red), más un F4 (latencia de E/S) no ejecutable. El análisis es descriptivo (medianas, rangos intercuartílicos e intervalos de confianza de mediana) más un único contraste de dos grupos intra-CNPG (F1 vs. F2). Este suplemento amplía las Secciones 5 y 6 del artículo.

S2 Metodología extendida

S2.1 Entorno

Kubernetes v1.34.6; PostgreSQL 16.13; CNPG 1.28.0; Chaos Mesh 2.8.3; Calico 3.31.4; almacenamiento CSI Huawei 4.10.1 (`csi.huawei.com`) sobre *Fibre Channel* (SAN); *StorageClass* con *Retain* y *WaitForFirstConsumer*. Entorno *air-gapped*: imágenes importadas localmente.

S2.2 Aislamiento (tres controles + malla)

El clúster experimental se fijó por afinidad al nodo de laboratorio, compartido con otras cargas. El aislamiento no descansó en la exclusividad del nodo sino en: (i) Chaos Mesh restringido al espacio de nombres; (ii) doble filtro de selección (espacio de nombres + nombre de clúster) en cada manifiesto de fallo; (iii) verificación previa (*dry-run*) que confirmó, para cada selector, resolución exclusiva a los pods del clúster experimental. Además, los pods se excluyeron de la malla (Linkerd) para no introducir un *proxy* en la ruta de datos que sesgara RTO/latencia.

Cuadro S1: Análisis de sensibilidad de la exclusión de F2 (arranque en frío).

Conjunto F2	n	Mediana (s)	Brecha F2−F1 (s)
Analizado (sin arranque en frío)	10	36,75	28,84
Con la 1.ª inyección (≈80,5 s)	11	36,92	29,01

Incluir el punto excluido eleva la mediana de F2 y *ensancha* la brecha respecto a F1 (mediana 7,91 s). La exclusión es, por tanto, conservadora.

S2.3 Verificador de transacciones (`tx-verifier`) y semántica ante ACK perdido

El verificador inserta identificadores BIGINT monótonos generados por un contador de cliente y anota cada COMMIT con marca de tiempo. Ante un fallo **reintenta el mismo identificador** (no avanza el contador), de modo que los cortes no crean huecos artificiales y un hueco en la tabla de verdad equivale a una escritura reconocida y perdida (proxy del RPO por contigüidad de la secuencia).

Esto resuelve el caso frontera de un COMMIT *confirmado pero no reconocido* (el primario persistió la fila pero el ACK se perdió por el corte): al reintentar el mismo id, un error de **clave duplicada** revela que la escritura *sí* había persistido ("commit fantasma"). El verificador detecta el duplicado y lo contabiliza como escritura persistida, no como pérdida; por eso el RPO no se sobreestima. El mecanismo está en el manifiesto `execution-package/manifiestos/30-workload/tx-verifier-cnpg.yaml` (la línea que reintenta el mismo id y detecta el duplicado como *commit* fantasma).

S2.4 Granularidad y precisión de medición

El verificador opera en bucle apretado sin espera entre escrituras exitosas; ante un fallo aplica una espera de 0,2 s y un `PGCONNECT_TIMEOUT` de 2 s. La **granularidad de medición del RTO** (hueco entre COMMIT consecutivos) es, por tanto, $\approx 0{,}2$ s. Las marcas de tiempo de COMMIT tienen resolución de milisegundos; la incertidumbre sistemática en el punto de recuperación es de $\pm 0{,}2$ s (la cadencia del sondeo). Las cifras a dos decimales deben leerse con esa incertidumbre.

S3 Escenarios, muestra y exclusiones

F1 y F2: $n = 10$ cada uno. F3: 10 inyecciones fijas de 60 s (usadas en el RTO) más 2 exploratorias (149 s y 300 s) excluidas del cálculo del RTO; no-promoción observada en las 12 (0/12). El tamaño muestral responde a las ventanas de mantenimiento del clúster productivo; no se hizo cálculo formal de potencia. Dada la no-normalidad esperada, se reportan medianas e IQR.

Exclusiones (transparencia). Se excluyeron: (a) la 1.ª inyección de F2 ($\approx$ 80,5 s, arranque en frío: caché fría, cola de reconciliación no calentada, primer ciclo de recreación); (b) las 2 inyecciones exploratorias de F3; (c) un identificador espurio negativo de un arranque previo. La exclusión (a) es **conservadora respecto al hallazgo principal**: incluir los 80,5 s *amplía* la brecha F1–F2 (Tabla S1), luego no favorece la tesis.

S4 Estadística completa

Todas las cantidades se reproducen con `analyze.py` (biblioteca estándar de Python).

Descriptivos. F1: mediana 7,91 s, IQR [7,62, 8,01], rango [6,61, 8,17]. F2: mediana 36,75 s, IQR [36,20, 37,06], rango [35,51, 38,32]. Razón de medianas F2/F1 $= 4{,}65\times$.

Intervalos de confianza de mediana (distribución-libre). Con $n = 10$ se usan los estadísticos de orden 2.º y 9.º, $[x_{(2)}, x_{(9)}]$, con cobertura

$$1 - 2\sum_{k=0}^{1}\binom{10}{k}0{,}5^{10} = 0{,}9785\ (\approx 97{,}9\%).$$

Resulta: F1 [6,85, 8,15] s; F2 [36,05, 37,66] s; F3 fija [60,74, 60,77] s.

Contraste F1 vs. F2. Mann–Whitney $U = 0$ (separación total, sin solapamiento). El p exacto bilateral es

$$p = \frac{2}{\binom{20}{10}} = \frac{2}{184\,756} = 1{,}083 \times 10^{-5},$$

que es el **piso teórico** alcanzable con $n = 10$ por grupo: el test está saturado y el peso probatorio recae en el tamaño del efecto y el mecanismo, no en la magnitud del p. Aproximación normal: $z = -3{,}78$ ($p \approx 1{,}6 \times 10^{-4}$). Correlación rango-biserial $= 1{,}00$. Diferencia de medianas resistente (Hodges–Lehmann) $= 28{,}96$ s.

Prueba de permutación (robustez ante no-independencia). Las 10 repeticiones se ejecutaron en serie; en F2 hay una asociación orden–RTO de magnitud moderada a fuerte (Spearman $\rho \approx 0{,}62$, valor crítico exacto para $n = 10$ a $\alpha = 0{,}05$ es 0,648; *no* significativa). El escenario está, además, completamente confundido con el bloque horario (F1 por la tarde, F2 por la noche), de modo que no existe esquema de permutación capaz de separar escenario y bloque: exigir que se respeten los bloques deja una sola reasignación admisible, la identidad. Por eso la conclusión se apoya en el tamaño del efecto y en el mecanismo, y no en el p.

Cotas binomiales (Clopper–Pearson). Cota superior **unilateral** al 95 % con 0 éxitos: $1 - 0{,}05^{1/n}$. F2 (0/10): $\leq 25{,}9\%$. F3 (0/12): $\leq 22{,}1\%$. (La cota bilateral al 95 % sería 30,8% y 26,5%.) La conclusión de no-promoción se apoya sobre todo en el mecanismo observado, no solo en el conteo.

RPO nulo: mecanismo. El RPO fue 0 en los tres escenarios; solo F1 lo pone a prueba de forma no trivial (hay promoción real). Con replicación asíncrona, un *pod-kill* podría en principio descartar WAL confirmado no propagado; que no ocurriera en 10/10 se atribuye a un retardo de replicación (*lag*) ≈ 0 bajo carga ligera (pgbench `-s 10`, 4 clientes). **Es una inferencia de mecanismo, no una medición**: no se registró el LSN *lag* durante las corridas (medición asignada a la Fase 2). La co-localización intra-nodo implica además que la replicación no atravesó red inter-nodo, por lo que el RPO=0 de F1 no prueba todavía la durabilidad ante partición o latencia inter-nodo reales.

S5 F4 (IOChaos / FUSE): bloqueo de instrumentación

El escenario F4 (sensibilidad del RTO a latencia de E/S de 0, 20, 50 y 100 ms vía IOChaos) **no fue ejecutable** sobre CNPG. El inyector FUSE `toda` de Chaos Mesh hizo *ptrace attach* sobre PostgreSQL, pero al montar el sistema de archivos FUSE falló con `Read-only file system (os error 30)` y entró en reintentos. Causa: CNPG endurece sus pods con `readOnlyRootFilesystem: true` (junto con `runAsNonRoot`

y descarte de capacidades), mientras que `toda` requiere una raíz escribible para su andamiaje de montaje. La configuración era correcta (el *dry-run* de selectores resolvió solo a los pods del clúster experimental; el `PodIOChaos` llevaba ruta, contenedor y parámetros correctos): es una **incompatibilidad**, no un error de configuración. Es una limitación de este par de herramientas, no una propiedad general. Dos vías para la Fase 2: (i) test confirmatorio desactivando `readOnlyRootFilesystem` para aislar la causa; (ii) limitación de ancho de banda de E/S a nivel de *cgroup* (`io.max`), que no depende de FUSE ni modifica el sistema bajo prueba. Los manifiestos de F4 se incluyen en el depósito para referencia.

S6 Reparto de responsabilidades por capa (Tabla S1)

Síntesis analítica del comportamiento documentado de los tres operadores; sustenta la discusión de la Sección 3.3 del artículo y *no* es una medición empírica.

Tipo de fallo	**Operador (*O*)**	**Kubernetes (*K*)**	**Almacenamiento CSI (*M*)**
Fallo del pod primario	Detecta la pérdida del líder y promueve una réplica (CloudNativePG: gestor de instancias propio; Zalando y Crunchy: Patroni)	Actualiza *Services* y *Endpoints* hacia el nuevo primario. CNPG gestiona los pods directamente, sin *StatefulSet*: es el operador, no el *kubelet*, quien crea el pod sustituto y lo incorpora como réplica	Reasocia el PVC existente al pod recreado; sin acciones sobre los datos
Fallo de nodo	Ordena la promoción de una réplica si el primario residía en el nodo afectado	Marca el nodo *NotReady* (≈50 s por defecto en Kubernetes v1.34, *node-monitor-grace-period*; la documentación de CNPG sitúa entre 40 y 55 s el intervalo hasta que el pod queda marcado como no listo) y desaloja los pods tras el umbral de tolerancia (≈5 min por defecto, *tolerationSeconds*) (Kubernetes, 2026a)	Local: datos inaccesibles hasta recuperar el nodo. En red o distribuido: un volumen RWO de un nodo caído no se desasocia hasta que vence el temporizador del controlador de *attach/detach* (≈6 min) o se aplica la marca `node.kubernetes.io/out-of-service`
Partición de red	Debe evitar el *split-brain*: cercado (*fencing*) del antiguo primario (Patroni: expiración del *lease* en el DCS, que en Kubernetes son objetos nativos —*Endpoints* o *ConfigMaps*— y no un etcd propio; CloudNativePG: estado reconciliado en el *API server*)	El *API server* actúa como punto único de coordinación; una minoría del plano de control bloquea la reconciliación	En red: E/S bloqueada o degradada; distribuido: la pérdida de quórum puede congelar las escrituras

Cuadro S2: Responsabilidad de detección y recuperación por capa y tipo de fallo.

Nota: síntesis analítica del comportamiento documentado por capa ante cada tipo de fallo; no es una medición empírica. *O* = operador, *K* = plano de control de Kubernetes, *M* = almacenamiento CSI. Fuente: documentación de CloudNativePG (CloudNativePG, 2026), Patroni (Patroni, 2026) y Crunchy (Crunchy Data, 2026); parámetros de desalojo de Kubernetes (Kubernetes, 2026a).

S7 Derivaciones trasladadas desde el artículo

Poder de detección del RPO

El `tx-verifier` escribe de forma estrictamente secuencial: cada inserción es una invocación independiente de `psql` que espera su confirmación antes de lanzar la siguiente, de modo que hay, a lo sumo, **una transacción en ejecución**. La pérdida máxima detectable por inyección es, por tanto, de *un* identificador, y solo si ese *commit* cae en la ventana entre el reconocimiento al cliente y el envío a la réplica. Con replicación asíncrona y retardo próximo a cero bajo la carga ligera empleada, la probabilidad de que esa ventana sea observable en una inyección dada es pequeña.

Lo que el piloto acredita es, pues, una cota y no una garantía. Con $x = 0$ pérdidas en $n = 10$ inyecciones válidas, la cota superior unilateral al 95 % de Clopper–Pearson es $1 - 0{,}05^{1/10} = 0{,}2589$, es decir, **25,9 %** para la probabilidad de pérdida por *failover*. Un sistema que perdiera una fracción apreciable de las transacciones elegibles podría haber pasado inadvertido en diez repeticiones. Añádase que **solo se verificaron las transacciones del verificador**: las de pgbench, con cuatro clientes concurrentes, no se comprobaron, de modo que la afirmación de ausencia de pérdida no se extiende a ellas.

Rejilla de muestreo dependiente del escenario (F3)

La nota de la tabla del artículo declara una granularidad de instrumento de $\approx 0{,}2$ s. Es correcta para F1 y F2, y **no** lo es para F3. El verificador espera 0,2 s tras un fallo y reintenta, pero su tiempo de espera de conexión es de 2 s (`PGCONNECT_TIMEOUT`). Cuál de los dos domina depende de cómo falle la conexión:

- En F1 y F2 el pod desaparece o deja de estar listo, el *endpoint* del servicio se retira y la conexión se *rechaza* de inmediato. El ciclo lo fija la espera de 0,2 s.
- En F3 el pod permanece *Ready*, el *endpoint* no se retira y la política *descarta* los paquetes en silencio. Cada intento agota los 2 s, y la rejilla efectiva pasa a ser de ese orden.

Por eso la indisponibilidad de F3 se reporta como indistinguible de la duración inyectada y no con dos decimales: la precisión aparente de las repeticiones no procede de la resolución del instrumento, sino de que lo medido es la duración de la propia política.

Conexiones establecidas y *conntrack* (F3)

Calico aplica la política a las conexiones *nuevas*; las ya establecidas sobreviven hasta que se reinicien o expiren, porque las cadenas de política del *workload endpoint* aceptan el tráfico ya seguido por *conntrack*. Felix ofrece una opción experimental para purgar ese estado al cambiar la política, lo que confirma que el comportamiento por omisión es el descrito (Calico, 2026). En F3 esto implica que la replicación en *streaming* y la vigilancia del gestor de instancias contra el *API server*, ambas establecidas antes de la inyección, siguieron con toda probabilidad activas. **Consecuencia para la Fase 2:** un aislamiento por iptables o eBPF seguirá siendo insuficiente si no purga el estado de *conntrack* o emplea reglas que no exceptúen las conexiones establecidas.

Ventana efectiva de F2

El manifiesto de F2 declara `duration: 10m`, pero el arnés (`run-experiment.sh`) retira la inyección en cuanto agota su bucle de detección de promoción, de 240 sondeos a 0,5 s ($\approx$120 s). Como en F2 nunca hay

cambio de primario, el bucle se agota siempre y la ventana efectiva es de unos dos minutos. La recuperación observada (36,7 s) precede con holgura a esa retirada, de modo que no es atribuible a ella.

S8 Diseño de la Fase 2

Lo que este piloto no pudo resolver queda asignado a una Fase 2 ya diseñada, que se ejecutará en un clúster multinodo dedicado y fuera de producción, en el marco del Trabajo de Especialización en Desarrollo de Software de la FaCyT-UC. Su requisito previo es un protocolo de validación del inyector: cada escenario deberá acreditar, mediante los eventos del operador y el estado del pod ante el plano de control, que indujo el fallo que nombra antes de que su medición se dé por válida. Sobre esa base se abordan seis cuestiones. F2 se repetirá con terminación directa del proceso (`container-kill` o `SIGKILL` periódico), sin tocar la especificación del pod, para contrastar el resultado con los 36,7 s de la reconciliación. F3 se reconstruirá con aislamiento completo por iptables o eBPF, que cierre también el tráfico de *host*, y permitirá ejercitar la comprobación `isolationCheck`. El fallo de nodo se probará con antiafinidad *obligatoria*, midiendo el desalojo y la reasociación del volumen. F4 se inyectará por `io.max` de *cgroups* v2, sin FUSE. El retardo de replicación se instrumentará registrando `pg_current_wal_lsn()` y `pg_last_wal_replay_lsn()` bajo una carga de escritura intensiva, para dar poder de detección al RPO. Y el contraste entre operadores y clases de almacenamiento, hoy solo analítico, pasará a ser empírico. Todo ello bajo un diseño factorial 2×2: replicación asíncrona frente a síncrona por quórum, y topología colocalizada frente a distribuida.

S9 Limitaciones extendidas

- **Un operador, una categoría de almacenamiento:** solo CNPG sobre CSI en red (SAN/FC). La comparación con Zalando/Patroni y Crunchy es analítica (Tabla 1 del artículo) y se asigna a la Fase 2.
- **Topología co-localizada:** las tres instancias residieron en un único nodo elegible; el *failover* es intra-nodo y un fallo de nodo completo no es *testeable* por construcción. El RTO de F2 **no** acota el real ante pérdida de nodo: CNPG es *shared-nothing* y la reasociación del volumen no está en el camino crítico del *failover*; además, F2 no reprodujo una indisponibilidad genuina (véase el artículo).
- **Carga pequeña:** pgbench `-s 10` ($\approx$150 MB), 4 clientes: no es un peor caso; central para el *lag* ≈ 0 y el RPO=0.
- **Solo asíncrono:** el modo síncrono no se varió; las cifras de RPO corresponden a replicación asíncrona.

Disponibilidad

Datos limpios, manifiestos, `tx-verifier`, scripts y este suplemento se depositan en Zenodo (DOI: 10.5281/zenodo.23004248, CC-BY-4.0; código MIT). Los registros crudos del verificador, las marcas del arnés y los eventos del plano de control no se conservaron: quedaron en el clúster productivo, que ya no está disponible. No pueden facilitarse. Por eso el mecanismo de F2 se presenta en el artículo como inferencia a partir de los manifiestos, no como observación.